How to use: paste this entire document into Claude (with the SportDrip repo attached) as the task prompt.

Prompt: Implement SportDrip's live-ops (rotating and promotional) engagement systems

Your role

You are a senior full-stack engineer on **SportDrip**, a social sports-prediction iOS app. Build the **infrastructure that lets the team run rotating and promotional content without app releases**: admin tooling, config-driven data models, schedulers, the client surfaces that read them, and the validators that enforce product and legal rules.

You are *not* writing the content itself (which items rotate, which art, which tournaments). Seed each system with clearly-labelled placeholder data so it works end to end, and document how operators fill it.

Repository context

- **Monorepo:**
  - `artifacts/mobile/` is the Expo SDK ~58 React Native app (expo-router, reanimated, Firebase JS SDK, TS strict; `@/` resolves to `artifacts/mobile/`).
  - `functions/src/` is Firebase Functions v2 (TS, zod, firebase-admin).
  - `firestore.rules` and `firestore.indexes.json` are at the repo root.
  - `scripts/` exists for node tooling, and a `service-account.json` is present for admin scripts. Never commit new secrets.
- **Typecheck:**
  - App: `artifacts/mobile/node_modules/.bin/tsc --noEmit -p artifacts/mobile`
  - Functions: `cd functions && ../node_modules/.bin/tsc --noEmit -p .`
- **Tests:** `cd functions && npm test`. **Baseline: 131 pass / 2 pre-existing failures** (`getPlayerProfile` uses `.select()`, which the fake Firestore lacks). Add no new failures.
- **Economy:** free credits, cosmetic-only spending, no cash-out, no purchase of wager credits. Catalog items live in `catalog/{itemId}` (`slot`, `price`, `rarity`, `available`, optional `levelReward`). Inventory is in `users/{uid}/inventory`, and purchases go through `callable/shop.ts`.
- **Already built that these systems plug into:**
  - **Prize boxes:** `PRIZE_BOXES` is hard-coded in `functions/src/lib/economy.ts`. The pity rule is in `callable/prizes.ts` (`PITY_AFTER`), and the published odds are mirrored in `artifacts/mobile/constants/prizeTypes.ts` (`BOX_ODDS`).

- **Weekly quests:** `functions/src/callable/quests.ts` (fixed `QUESTS` array, XP rewards via `grantXp`).
- **Daily Pick:** `functions/src/callable/dailyPick.ts`. `chooseDailyPick` respects an existing `dailyPick/{day}` doc, so admin pre-writes override it.
- **Notifications and push:** `writeNotification` + `simpleNotification` (`functions/src/lib/notify*.ts`) feed `triggers/push.ts`, which sends Expo pushes on doc create. Notification types are a union on server and client, and `app/activity.tsx` has an icon map that must include every type.
- **Static engines (may exist by the time you run; check first):** season pass engine (`meta/seasons/*`, `seasonXp`), bracket mode (`bracketTemplates/*`), league co-op goals (`meta/coopRewards`), Drip Wrapped generator, experiments (`meta/experiments`, `lib/experiments.ts`), and the push budget / quiet hours in `triggers/push.ts`.
- **If a static engine is missing:** build only the minimal hook your item needs, and flag it in the report. Do not build the full static feature here.

Non-negotiable guardrails (enforce in code, not just copy)

1. **Never sell wager credits, and nothing earned with credits converts to real-world value.** Real-world prizes live in a completely separate, free-entry promotion flow (item 11).
2. **Odds transparency:** any random reward table (prize boxes, cosmetic drops) is a *single* config doc that both the server roll and the client odds display read. Changing it changes the published odds atomically. The pity guarantee always applies.
3. **No fake scarcity:** - "Limited" items have a real `endsAt` that the server enforces and the UI shows. - No countdown nags or confirmshaming. - A rotation is predictable (weekly), not daily FOMO.
4. **Season pass:** never replaces lifetime levels. A premium track priced in credits can never pay out enough credits to buy the next pass, enforced by a validator that rejects the config.
5. **Push campaigns:** - Respect the per-user daily budget, quiet hours and marketing opt-in. - Never target users on a break, at their daily limit, or within 12h of a loss with "come back and bet" messaging. - Live Activity / widget content is never promotional.
6. **Sweepstakes / prizes:** - free to enter, with official rules - "Apple is not a sponsor" disclaimer - an eligibility geo-gate excluding restricted states (configurable list; default CA, NY, NJ, CT, MT) - an 18+ check - no connection to wallets, bets or credits
7. Every content edit is validated (zod) **before** it is written. Invalid config must never reach clients.

Engineering conventions

- **Config docs:**
  - Live under `meta/*`, or a dedicated collection, with a zod schema shared between the admin tool and functions (put schemas in `functions/src/lib/liveops/schemas.ts` and import them in tooling).
  - Firestore rules allow client **read** of published config only, and **no client writes**.

- **Admin writes:**
  - Go through admin-only callables guarded by a custom claim (`admin: true`) set with a script, or through the CLI using the service account.
  - Every admin write appends to `liveopsAudit/{id}` (who, what, before/after, when).
- **Scheduling:** time-based activation reads `startsAt` / `endsAt` server-side. Clients never decide what is active from their own clock.
- Keep the UI on the existing kit (`components/ui`) and `useColors()` tokens, with accessibility labels on every pressable.
- Add unit tests in `functions/test/` for validators, selection logic (rotation pick, quest selection by week hash) and eligibility gates.

The work

Build the prerequisite first, then the items. The section text below is the operating spec. For each item, implement its **data model, server logic, admin tooling, client surface and validators**, then seed placeholder data.

One-time prerequisite: admin tooling. Every item below is driven by Firestore config docs (`meta/*`, `catalog/*`, `bracketTemplates/*`). Build a single admin surface (a protected Forge/web page or a `scripts/` CLI using the service account) with schema validation per doc. Content edits should never need an app release.

---

1. Weekly featured shop rotation (A13)

- **What:** a predictable weekly set of featured items, with wishlist alerts when a wished item rotates in. Avoid daily FOMO churn.
- **Data:** `meta/shopRotation/{weekKey}` = `{ featuredItemIds[], startsAt, endsAt }`. Wishlists live at `users/{uid}/wishlist/{itemId}`.
- **Code hook:** the shop reads the current week's doc. A Monday job sends a `wishlist_available` notification as an **engagement** push, so it is subject to the budget.
- **Ops:** plan 4 weeks ahead; mix rarities.
- **Rule:** "Limited" means truly limited, with a real `endsAt` shown in the UI.

2. Calendar drops (rivalry week, playoffs, opening day)

- **What:** themed items tied to the sports calendar.
- **Data:** catalog items with `availableFrom` / `availableUntil`, plus an optional `eventTag`.
- **Ops:** keep a sports calendar sheet, and create the items and art 2–3 weeks ahead.
- **Rule:** no countdown nags. The prize-box card's "never a countdown nag" stance applies.

3. "Live" team jerseys

- **What:** a jersey gains trim when that team wins that week.
- **Data:** an item flag `liveTeam: "NFL:KC"`. A weekly job writes `teamForm/{teamKey}.wins` and the renderer picks the trim level from it.
- **Ops:** choose which teams get live jerseys each season and review the art for each trim level.

4. Cosmetic drops in prize boxes

- **What:** boxes can drop cosmetics as well as credits.
- **Data:** `meta/boxTables/{tier}` holds the segments (credits **or** itemId) with weights. The server and the client odds display read the same doc. Today's tables are hard-coded and mirrored, so move them to config before this item.
- **Ops:** every table change must update the published odds at the same moment (one doc, so this is automatic) and keep the pity guarantee.
- **Rule:** never sell boxes or wager credits for money. Odds stay visible.

5. Season pass content (A12)

- **What:** the free cosmetic track for each season (calendar month or league season).
- **Data:** `meta/seasons/{seasonId}` holds tiers, XP thresholds and reward item ids. The engine validates the doc.
- **Ops:** author the tiers and rewards one season ahead, and announce at season start (in-app banner, push within budget).
- **Premium track:** only if priced in credits, and the pass may **never** return enough credits to buy the next one (the validator enforces this). No real-money pass that grants wager credits.

6. Weekly quest pools

- **What:** rotate the 3 weekly quests from a pool instead of fixed definitions.
- **Data:** `meta/questPools` holds effort, social and skill pools with XP values. The week's selection is deterministic (`hash(week)`) so everyone sees the same set.
- **Ops:** add or retire quests monthly.
- **Rule:** every quest must point at the core loop (picks, H2H, social acts). The research found gamification unrelated to core value was neutral.

7. Co-op goal rewards (A15)

- **What:** the league-exclusive item unlocked by the weekly co-op goal.
- **Data:** `meta/coopRewards.current` = itemId, rotated monthly.
- **Ops:** read the experiment results before scaling; the research has no effect size for this.

8. Tentpole brackets (A17)

- **What:** March Madness and other tournaments as an acquisition and invite moment.

- **Data:** a `bracketTemplates/{id}` per event (slots, feeds, provider game-id mapping, lock time).
- **Ops:**
  - Build the template once the field is set (Selection Sunday for March Madness).
  - Pre-announce a week ahead.
  - Make "create a bracket league and invite" the primary CTA.
  - Monitor settlement mapping each round.
- **Calendar:** NCAA tournament, NFL playoffs, NBA playoffs, World Cup / Euros when relevant.

9. Drip Wrapped campaigns (A16)

- **What:** the yearly (December) and season-end recaps, generated statically, plus the campaign around them.
- **Ops:**
  - In-app banner and one push to lapsed users (the reactivation lever).
  - Share-card copy review.
  - Track shares per recap as the KPI.
  - Keep copy free of "bet more" messaging.

10. Push campaigns and reactivation

- **What:** occasional engagement pushes (new season, Wrapped, tentpole launch).
- **Data/hook:** a campaign doc `meta/campaigns/{id}` with audience (segment query), copy, send window and arm (experiment). The sender respects the per-user budget, quiet hours and marketing opt-in (Apple 4.5.4).
- **Rule:** never "come back and bet" after a loss; never to users on a break or at a daily limit.

11. Promotions with real-world prizes

- **What:** any giveaway with a tangible prize.
- **Must be:**
  - separate from credits (no credits → prize path)
  - free to enter
  - backed by official rules and a developer-sponsored "Apple is not a sponsor" disclaimer
  - geo-checked against states with sweepstakes restrictions (CA AB 831, NY, NJ, CT, MT and new 2026 bills)
- **Ops:** legal review per promotion; consider a third-party sweepstakes administrator.
- **Code:** a standalone entry flow, never connected to the wallet or bets.

12. Daily Pick editorial override

- **What:** featuring a marquee game over the automatic choice.
- **Data:** `dailyPick/{day}` can be pre-written by admin tooling; `chooseDailyPick` already respects an existing doc.

- **Ops:** use it for tentpoles (Super Bowl, Game 7s, rivalry week).

13. Experiment calendar

- **What:** which experiments run, at what split, for how long.
- **Data:** `meta/experiments` holds arms and splits only; treatments are code.
- **Ops:**
  - Run each test across full sports weeks.
  - Size for 2–5% relative effects.
  - Keep the global holdout permanently.
  - Randomise by league for social features.
  - Read results by intent-to-treat for anything users opt into (widgets, Live Activities).

Cadence at a glance

Cadence	Items
Daily	Daily Pick override (as needed)
Weekly	Shop rotation, quest selection (automatic), co-op goal, live-jersey refresh
Monthly	Season pass content, co-op reward, quest pool review
Seasonal / event	Calendar drops, tentpole brackets, Wrapped, push campaigns
Per promotion	Legal review, official rules, geo-gating

Implementation specifics per item (in addition to the spec above)

- **Prerequisite: admin tooling.** Build both of these, sharing the same schemas:
  - A CLI at `scripts/liveops.ts` (`list | get | validate | set --file x.json | schedule`).
  - Admin-only callables `liveopsGet` / `liveopsSet`.

  Also add a minimal admin screen in the app behind the `admin` claim: list config docs, view JSON, validate, publish. No WYSIWYG. - **Item 1 (shop rotation):** - Make the shop read `meta/shopRotation/{weekKey}` (week key from `functions/src/lib/weekly.ts`). - Add `users/{uid}/wishlist/{itemId}` with a heart toggle on item cards. - A Monday scheduled function sends `wishlist_available` notifications (new notification type on both sides, plus an activity icon), counted as **engagement** pushes. - **Item 2 (calendar drops):** - Add

`availableFrom` / `availableUntil` / `eventTag` to catalog items. - Enforce the window in `purchaseItem`, and hide out-of-window items in the shop. - **Item 3 (live jerseys):** - An item flag `liveTeam: "SPORT:ABBR"`, plus a weekly job writing `teamForm/{teamKey}` (wins this week) from final games. - The avatar renderer chooses the trim level from `teamForm` and caches it client-side. - **Item 4 (box tables):** - Move `PRIZE_BOXES` into `meta/boxTables/{tier}`, keeping the code values as the fallback seed. - Segments may be `{ credits }` or `{ itemId }`. An item drop grants inventory, or credits equal to its sell value if already owned. - The client odds table reads the same doc; delete the hard-coded `BOX_ODDS` mirror. - The validator requires weights > 0 and the top segment to stay reachable for pity. - **Item 5 (season content):** a validator plus a seed season, and a "Season" banner on Home while one is active. - **Item 6 (quest pools):** - Replace the fixed `QUESTS` with `meta/questPools`. - The weekly selection is `hash(weekKey)`, deterministic and identical for everyone, with one quest per pool plus the endowed check-in. - Keep progress evaluation in code: each pool entry references a known `metric` id that the server already computes. - **Items 7–9 (co-op rewards, brackets, Wrapped):** admin flows to set the reward id, create or validate a bracket template (with provider game-id mapping and lock time), and schedule the Wrapped banner and push. - **Item 10 (campaigns):** - `meta/campaigns/{id}` = `{ audience: segmentId, copy, sendWindow, experimentArm? }`. - Segments are defined in code (`lapsed_7d`, `in_league_no_pick_today`, etc.) so audiences are safe and testable. - A scheduled sender runs in batches, respects the budget, quiet hours and guardrail 5, records sends, and supports dry-run. - **Item 11 (promotions):** - A standalone `promotions/{id}` with official-rules URL, eligibility (states, age), start/end and entry cap. - An `enterPromotion` callable validates eligibility, and entries live in `promotions/{id}/entries/{uid}`. - A separate screen with the disclaimer. It never reads or writes wallets. - **Item 12 (Daily Pick override):** an admin action that writes `dailyPick/{day}` ahead of time, validating that the game exists and starts that day. - **Item 13 (experiment calendar):** an admin view of `meta/experiments`, including the start/end dates, the analysis unit (uid vs leagueId), and the guarantee that the global holdout is never edited away.

How to work

1. Read the files each item touches before editing, and follow the actual code where it differs from this prompt (note the difference in your report).
2. Use a task list. After each item, run both typechecks, and run `npm test` after server changes.
3. Seed placeholder content via the CLI with obviously fake names (e.g. "PLACEHOLDER Week 1 Featured").
4. Ask only for genuine product or legal decisions (restricted-state list, campaign copy). Otherwise pick the conservative default and record it.
5. Do not commit or push unless asked. Do not modify the pre-existing failing tests.

Final report

- **Per item:** done / partial / skipped, files changed, and deviations with reasons.
- **Operator guide:** for each system, the exact CLI commands or admin-screen steps to publish next week's content, plus the cadence (daily / weekly / monthly / per event).
- **Deploy checklist:**
  - functions, rules, indexes (check for duplicates)
  - how to grant the `admin` custom claim
  - config docs to seed
- **Test results:** typecheck output for both packages, and pass/fail counts versus the 131/2 baseline.